

Мови програмування

У проєкті використовується лише одна основна мова програмування — **JavaScript**. Завдяки обраному стеку, нею написано і сервер (на базі Node.js), і візуальна частина інтерфейсу (на React з використанням JSX). Додатково застосовуються стандартні вебтехнології — HTML та CSS.

JS (JavaScript): Основна мова програмування проєкту, яка відповідає за інтерактивність, обробку даних та логіку роботи як у браузері, так і на сервері.

JSX: Спеціальне розширення синтаксису JavaScript, яке дозволяє зручно писати HTML-розмітку безпосередньо всередині коду для створення компонентів екрана.

HTML: Базова мова розмітки, яка створює структурний каркас сторінки (тексти, кнопки, поля вводу).

CSS: Мова стилів, яка відповідає за візуальне оформлення сайту: кольори, шрифти, відступи та анімації.

React: Популярна JavaScript-бібліотека для швидкого створення інтерфейсів користувача з окремих блоків-компонентів без перезавантаження сторінки.

Node.js: Програмне середовище, яке дозволяє запускати код JavaScript на стороні сервера для обробки запитів та роботи з базою даних.

База даних

Чому SPA (React)?

Миттєва робота інтерфейсу без білих екранів завантаження — критично важливо під час паніки.

База даних

«MongoDB — це сучасна нереляційна база даних. На відміну від класичних баз, вона зберігає інформацію не у вигляді жорстких таблиць (як в Excel), а у вигляді гнучких текстових документів. Я обрала її, оскільки такий формат дозволяє набагато зручніше та швидше зберігати складні терапевтичні сценарії, щоденники та результати тестів користувачів без створення громіздких схем».

Чому було обрано для розміщення вебсистеми Render?

Я обрала Render, тому що на сьогодні це найзручніший і найстабільніший безкоштовний хостинг для таких проєктів. Він автоматично оновлює сайт,

щойно я завантажую новий код у GitHub, чудово підтримує всі наші технології та дозволяє надійно сховати секретні паролі від бази даних. Для дипломної роботи це ідеальний баланс швидкості та безпеки

UX/ UI

UX — это скелет и логика, понимание того, *как работает* система.

UI — это внешний вид и оболочка, понимание того, *как выглядит* продукт.

У чому наукова новизна проєкту?

Впровадження динамічної архітектури, що змінює інтерфейс під поточний психологічний стан.

Які методи дослідження застосовано?

Системний аналіз ринку, алгоритмічне моделювання UX та функціональне тестування.

Яку загальну архітектуру ви використовували для створення веб-системи?

«Ми використовували класичну клієнт-серверну архітектуру на базі сучасного стеку MERN (MongoDB, Express.js, React, Node.js).

Чому обрано MERN-стек?

Це універсальний набір технологій на одній мові, що спрощує взаємодію між клієнтом та сервером.

Що таке MERN

MongoDB — нереляційна база даних для зберігання контенту у вигляді гнучких документів.

Express.js — серверний фреймворк для обробки запитів та керування маршрутами.

React — бібліотека для побудови інтерфейсу користувача (фронтенду) з окремих блоків-компонентів.

Node.js — програмне середовище, що дозволяє виконувати JavaScript на стороні сервера.

Що таке API

API — це програмний посередник, який регламентує правила взаємодії між клієнтом (інтерфейсом користувача) та сервером (базою даних і логікою). Він приймає запит у чітко визначеному форматі, виконує задану операцію на стороні сервера та повертає відповідь, приховуючи всю складність фонових процесів від кінцевого користувача.

Якщо сайтом користуватимуться 100 осіб одночасно, сервер "ляже"?

Як вирішено проблему?

Ні, не ляже. 100 користувачів — це мізерне навантаження для нашого стеку. Сервер на Node.js працює повністю асинхронно і не блокується при одночасних запитах, а база даних використовує індекси, що дозволяє їй видавати інформацію за мілісекунди

Як рахується психологічний індекс?

Як середнє значення відповідей користувача за п'ятьма критеріями стресу та стану.

Чим MongoDB краща за класичні бази?

Гнучкістю формату документів, що дозволяє зберігати складні терапевтичні сценарії без жорстких таблиць.

Чому MongoDB ?

«Тому що вона ідеально сумісна з нашим стеком технологій (JavaScript/Node.js), дозволяє зручно зберігати складні вкладені дані (сценарії, статистику) без купи таблиць, і дає змогу гнучко змінювати структуру бази під час розробки».

У базі даних зберігаються два типи інформації:

1. Контент: навчальні матеріали, сценарії тренажерів, поради та питання для тестів.

2. Дані користувачів: акаунти (логін/пароль), рівень прогресу (стійкість), історія пройдених квестів, результати тестів та записи у щоденнику».

Як саме розраховується результат тесту на бекенді? Опишіть логіку контролера.

1. Отримання даних: Контролер (через POST-запит) отримує з фронтенду масив числових відповідей користувача.
2. Розрахунок бала: Сервер вираховує загальний результат як середнє арифметичне всіх відповідей (сума всіх відповідей / їхню кількість).
3. Обчислення мультиплікатора та збереження: На основі отриманого бала контролер розраховує коефіцієнт стійкості. Якщо бал менше 50, коефіцієнт знижується (щоб адаптивна система приховала складні квести). Якщо більше 50 — підвищується. Після цього результат записується в колекцію `DiagnosticResult`, а статистика користувача оновлюється.

Як працює «гостьовий режим»?

Запити до бази даних блокуються на рівні браузера, тому нічого не зберігається на сервері.

1. У `localStorage` браузера зберігається лише проста позначка-прапорець `dr_token: "guest_mode"`, щоб фронтенд знав, що потрібно відображати інтерфейс для гостя.
2. Але весь сам прогрес гостя (прочитані статті, статистика) прив'язаний до **Cookies**, які передаються разом із запитом (`credentials: "include"`).

Як захищені паролі користувачів?

Вони перетворюються на унікальний зашифрований код (хеш) ще до моменту запису в базу даних.

Чи зберігаються паролі в базі даних?

Так, але виключно у вигляді незворотного хешу (через бібліотеку `bcrypt`). Ми ніколи не зберігаємо паролі у відкритому вигляді з міркувань безпеки користувачів

Яка бібліотека шифрування?

Для хешування паролів ми використовували бібліотеку `bcryptjs`

Що якщо БД зламують?

Паролі зашифровані. Без JWT-токенів прив'язати результати до конкретної людини неможливо.

Де зберігаються важкі файли?

Вони зберігаються локально на самому сервері — у статичній папці додатку `public` (у підпапках `audio` та `images`).

Для чого був зроблений перемикач теми з темної на світлу?

Перемикач зроблено для комфорту. Хоча темна тема краще заспокоює, світла тема залишена для користувачів, яким важко читати світлий текст на темному тлі в денний час.

Чому це SPA-сайт, а не звичайний сайт?

Щоб інтерфейс працював миттєво, без очікування перезавантаження сторінок під час паніки.

Чи може ваша система замінити реального психолога?

Ні, такої мети не стояло. Наша платформа — це інструмент самодопомоги, базової рефлексії та екстреної підтримки. Для серйозних проблем система радить звернутися до спеціаліста

Як у вас реалізовано видання рекомендацій?

Це штучний інтелект чи жорсткі правила? «Це жорсткі правила (алгоритми). Система аналізує поточний бал стійкості та результати діагностики і за чіткою математичною логікою фільтрує контент, показуючи лише те, що підходить під цей бал».

Звідки були взяті іконки для сайту?

Ми використовували популярну бібліотеку "Lucide React". Вона містить сучасні SVG-іконки, які ідеально підходять під наш мінімалістичний дизайн і не перевантажують сайт

Темний фон допомагає для покращення психологічного стану?

Так, темні та приглушені кольори знижують зорове та когнітивне навантаження. Це дуже важливо для людей у стані стресу чи тривоги — такий інтерфейс менше подразнює нервову систему

Чи був проведений SEO-аналіз?

Так, базовий аудит доступності, продуктивності та SEO проводився за допомогою вбудованого інструменту Google Lighthouse у браузері.

Чи є блок самого акаунту?

Ні, класичної сторінки профілю з налаштуваннями чи аватарками ми не робили. Оскільки додаток фокусується на психологічній допомозі, замість типового профілю ми реалізували велику сторінку "Статистики". Саме там зберігається все, що важливо для користувача: його поточний рівень стійкості, історія проходження квестів та графік прогресу

Ви заявляєте "адаптивну" систему. Ви використовували готові фреймворки для дизайну або писали стилі вручну?

Писали вручну

«Ми використали комбінований підхід.

Для загальної адаптивної сітки ми використали сучасний фреймворк ****Tailwind CSS****.

Проте всі складні анімації, ефекти світіння, прозорості та специфічний дизайн самих тренажерів (наприклад, анімації перетягування карток) ми писали ****повністю вручну чистим CSS****, щоб отримати унікальний дизайн, який не схожий на шаблонні сайти».

На основі яких методик проводиться "оцінювання"? Ви самі вигадали тести?

Ні. В основу тестів закладено 5 клінічних критеріїв оцінки стану (рівень тривожності, порушення сну, емоційне виснаження, концентрація, та соматична реакція на стрес). Тексти питань ми адаптували з відкритих

стандартизованих психологічних опитувальників, щоб вони звучали просто і зрозуміло для звичайної людини».

Як ви перевіряли працездатність своєї системи?

1. Ручне функціональне тестування: Ми власноруч проходили всі квести та діагностику від імені різних користувачів (з низьким та високим рівнем стійкості), щоб переконатися, що алгоритм правильно приховує або показує потрібний контент.
2. Тестування адаптивності: За допомогою інструментів розробника (Chrome DevTools) перевіряли, як інтерфейс поводить себе на екранах різних розмірів — від маленьких смартфонів до великих моніторів.
3. Тестування безпеки: Перевіряли форми реєстрації та авторизації на спроби введення некоректних даних (наприклад, дублювати email або занадто короткі паролі), щоб впевнитись, що бекенд їх надійно блокує».

Як система знижує когнітивний шум?

Через темну колірну гаму, модульну сітку та автоматичне приховування зайвих кнопок.

Чи несе система медичну відповідальність?

Ні, це інструмент самодопомоги, про що вказано у правовому попередженні.

Як працює SOS-кнопка?

Вона накладається поверх усього екрана з найвищим пріоритетом, щоб відкритися миттєво.

Чи можна видалити всі свої дані?

Так, реалізовано повне каскадне стирання історії записів та тестів за запитом користувача.

Чому обрано «скломорфізм» у дизайні?

Для створення фокусу на елементах допомоги при високій швидкості роботи інтерфейсу.

Як система уникає повторів у тестах?

Завдяки алгоритму випадкової вибірки питань з великого списку при кожному новому тестуванні.

Яка практична значимість роботи?

Надання безбар'єрного та доступного інструменту для первинної психологічної стабілізації.

У чому полягає адаптивність додатку

Адаптивність працює так: система динамічно змінює контент залежно від рівня вашої стійкості та діагностики.

На Дашборді, в Медіатеці та в Квестах при падінні стійкості складні завдання і статті ховаються, а на їхньому місці одразу з'являються швидкі вправи для зняття стресу.

ПОВНЕ ПОЯСНЕННЯ

1. Головний дашборд (блок «Рекомендовано для вас»): Тут автоматично підміняються швидкі картки. Якщо стійкість низька — з'являються вправи на дихання. Якщо висока — картки для розвитку.
2. Медіатека: Повністю фільтрується список статей. При стресі зникають довгі матеріали (наприклад, про лідерство) і залишаються лише інструкції для важких станів (що робити при паніці).
3. Розділ Квестів (Завдання): Система підбирає наступні місії залежно від того, як ви пройшли діагностику та який у вас зараз відсоток стійкості.

Чому обрали саме такі кольори

Кольори підібрані так, щоб інтерфейс заспокоював, а не дратував. Темний фон знижує візуальний шум і навантаження на очі під час стресу. При цьому яскраві акценти використовуються строго функціонально: наприклад, червоний колір виділено тільки для кнопки SOS, щоб людина інтуїтивно і миттєво знаходила її в стані паніки, не відволікаючись на інші елементи.

Использовали ли вы какие-то паттерны проектирования на фронтенде (React)?

«Так, ми використали 4 основні патерни React:

1. Компонентний підхід — розбиття інтерфейсу на дрібні незалежні блоки (кнопки, картки), щоб їх можна було перевикористовувати.
2. Поділ на "розумні" та "дурні" компоненти (Container/Presentational) — коли головний компонент керує логікою і даними, а дочірні просто відповідають за їх красиве відображення.
3. Патерн Хуків (Hooks) — використання `useState` та `useEffect` для керування станом.
4. Умовний рендеринг — саме завдяки цьому патерну наша система "на льоту" приховує або показує контент залежно від рівня стійкості».

1. Код для зміни розміру/кольору кнопки при наведенні:

«Для цього використовується CSS-псевдоклас `:hover` у комбінації з властивістю `transition` (для плавності). Зміна розміру робиться через `transform: scale(1.05)` (збільшення на 5%), а зміна кольору — просто заміною `background-color`. У нашому проекті ми часто використовували класи фреймворку Tailwind, наприклад: `hover:scale-105 transition-all`.»

2. За допомогою якого коду був зроблений перемикач (тумблер):

«Логіка перемикача написана на React за допомогою хука стану `useState`. Клік по перемикачу змінює стан (State) з `false` на `true`. А візуальний ефект (коли кружечок "перекочується") зроблено через CSS-властивість `transform: translateX()`, яка зсуває елемент по горизонталі.»

3. Анімація для кнопки «назад»:

«Вона реалізована за допомогою CSS-трансформацій. При наведенні курсору (`:hover`) до кнопки застосовується `transform: translateX(-5px)` (або `translateY`), що змушує її трохи зміститися вліво чи вгору, імітуючи рух. Плавність забезпечує властивість `transition: all 0.3s ease`.»

4. Який код було використано для екстреної допомоги (Кнопка SOS)

: «Сама кнопка прив'язана до екрану за допомогою CSS-властивості `position: fixed` — тому вона не зникає при прокручуванні. На ній висить React-обробник подій `onClick`, який при натисканні миттєво відкриває модальне вікно (змінюючи стан `showSOSModal` на `true`) з диxальними вправами, перекриваючи весь інший контент.»

5. Як було зроблено вхід та реєстрація:

«Фронтенд (React) збирає дані з форми і відправляє POST-запит на бекенд (Node.js). Під час реєстрації сервер бере пароль, криптографічно його шифрує (хешує) за допомогою бібліотеки `bcryptjs`, і лише після цього зберігає запис у базу даних MongoDB.»

6. Як був зроблений вхід саме через пошту:

«Коли користувач вводить пошту та пароль, бекенд шукає в MongoDB обліковий запис із таким email. Якщо знаходить — перевіряє, чи співпадає хеш введеного пароля з тим, що в базі. Якщо все правильно, сервер генерує JWT-токен (JSON Web Token) і відправляє його на фронтенд. Фронтенд зберігає цей токен і використовує його як "перепустку" для всіх наступних дій на сайті.»

Як була зроблена аналітика?

1. Коли користувач щось робить (наприклад, проходить тест), додаток передає цю інформацію на сервер. Сервер за формулою вираховує відсоток стійкості і зберігає його в базу даних. А коли ви заходите в аналітику, спеціальна бібліотека просто бере ці готові збережені цифри з бази і малює з них графік

Як було зроблено підвищення звуку?

Завдяки Бібліотеці Web Audio API який вміє легко плавно нарощувати гучність

Чому не по стандарту?

Щоб зробити плавне підвищення гучності зі стандартний тегом <audio> , довелося б писати "милиці" , які штучно змінювали б гучність кожні кілька мілісекунд

Як був зроблений помічник?

1. Візуального помічника немає на сторінці постійно: в потрібний момент фреймворк React просто вставляє його в HTML-код сторінки, а потім так само вирізає

2. С кодом ответ:

механізм умовного рендерингу в React, код помічника фізично вставляється , коли змінна стану isVisible змінюється на true. Рівно через 10 секунд спрацьовує вбудований таймер, який змінює цю змінну назад на false, змушуючи вирізати HTML-код помічника з браузера.

Як був зроблений щоденник?

«Щоденник працює за класичною клієнт-серверною моделлю. Фронтенд (React) збирає написаний текст та обрану емоцію і відправляє їх на сервер. Сервер зберігає цей запис у базі даних. При наступному відкритті сторінки додаток просто забирає готову історію записів із бази і виводить її на екран».

Як реалізован скролл

Відновлення скролу реалізовано шляхом постійного збереження поточної позиції у sessionStorage через подію onScroll унікально для кожного розділу. При поверненні до сторінки програма зчитує це значення з пам'яті та автоматично повертає екран на потрібне місце за допомогою плавного методу scrollT з параметром smooth

Як була зроблена анімація для кнопки «назад»?

«Анімація реалізована через 3D CSS-трансформації. У потрібний момент кнопка фізично розвертається на 180 градусів, показуючи свою приховану зворотну сторону з написом "Назад".

А щоб уся панель переверталась хвилину (ефект доміно), до кожної наступної кнопки додається невелика математична затримка у 0.1 секунди».

Як була зроблена анімація для кнопки «назад»?

«Анімація працює через 3D CSS-властивість rotateY(180deg). Вона розвертає кнопку зворотною стороною, де прихований напис "Назад".

Послідовність перевертання кнопок реалізована через transition-delay: скрипт просто додає кожній наступній кнопці 0.1 секунди затримки».

Як був зроблений пошук?

Пошук реалізовано максимально просто і швидко:
він працює через одну стандартну JavaScript-функцію .filter()

Як був зроблений сад стійкості?

«Технічно сад стійкості — це звичайна векторна SVG-іконка. Жодної важкої графіки: React просто підміняє один SVG-файл на інший залежно від прогресу, а CSS-властивість transform: scale() динамічно змінює фізичний масштаб картинки».

Як працюють рекомендації на дашборді?

«Коли ви обираєте настрій, скрипт "на льоту" шукає в назвах карток відповідні ключові слова. Якщо таких знайдено мало, система просто доповнює список випадковими матеріалами».

Як було проведено SEO-оптимізацію?

«SEO-оптимізація в проєкті побудована на чіткій семантичній структурі HTML5. Замість того, щоб робити все через однакові теги <div>, інтерфейс розмічений правильними структурними тегами (<main>, <section>, <nav>, правильна ієрархія заголовків <h1> - <h3>). Пошукові роботи легко "читають" таку структуру. Крім того, висока швидкість завантаження коду і правильні метатеги дозволили отримати максимальні 100 балів за SEO в аудиті від Google Lighthouse».

БАЗОВІ ПИТАННЯ ПО КОДУ

Як зробити так, щоб елементи вишикувалися в ряд по горизонталі, а не падали один під одного?

flex-direction: column для колонки та flex-direction: row для рядка

Як у React вставляються зображення? Ви писали звичайний тег `<img>`?

Відповідь: «Так, використовується тег `<img>`, але шлях до зображення вказується інакше. Спочатку я імпортувала зображення на початку файлу (наприклад, `import myIcon from '../assets/icon.png'`), а потім вставляла змінну в атрибут `src` у фігурних дужках: `<img src={myIcon} alt="іконка" />`».

Як технічно з'являються бічне меню та спливаючі вікна?

3. Питання-пастка: «Node.js — це ваша база даних?»

Ні. База даних — це MongoDB, саме там фізично лежать тексти, тести та користувачі.

Node.js — це програмне середовище, на якому працює мій сервер.

Щоб відкрити консоль розробника (DevTools)

`⌘ Option + ⌘ Command + I`